language, essentially Business Domain vernacular, and the end result is simpler, more readable test scripts. Figure 2 makes it easier to visualise this. Inside our chosen IDE, we create a script file, which is simply a text file with a dot robot *(.robot)* extension. These script files reference workflow files, which are lower level keywords, as well as text *.robot* files. Those workflows themselves make calls to the various libraries. So, we can see the libraries we have access to include built-in libraries that come with the Robot Framework itself, and also external libraries that can be installed by using *pip3 install* and then the name of the library.

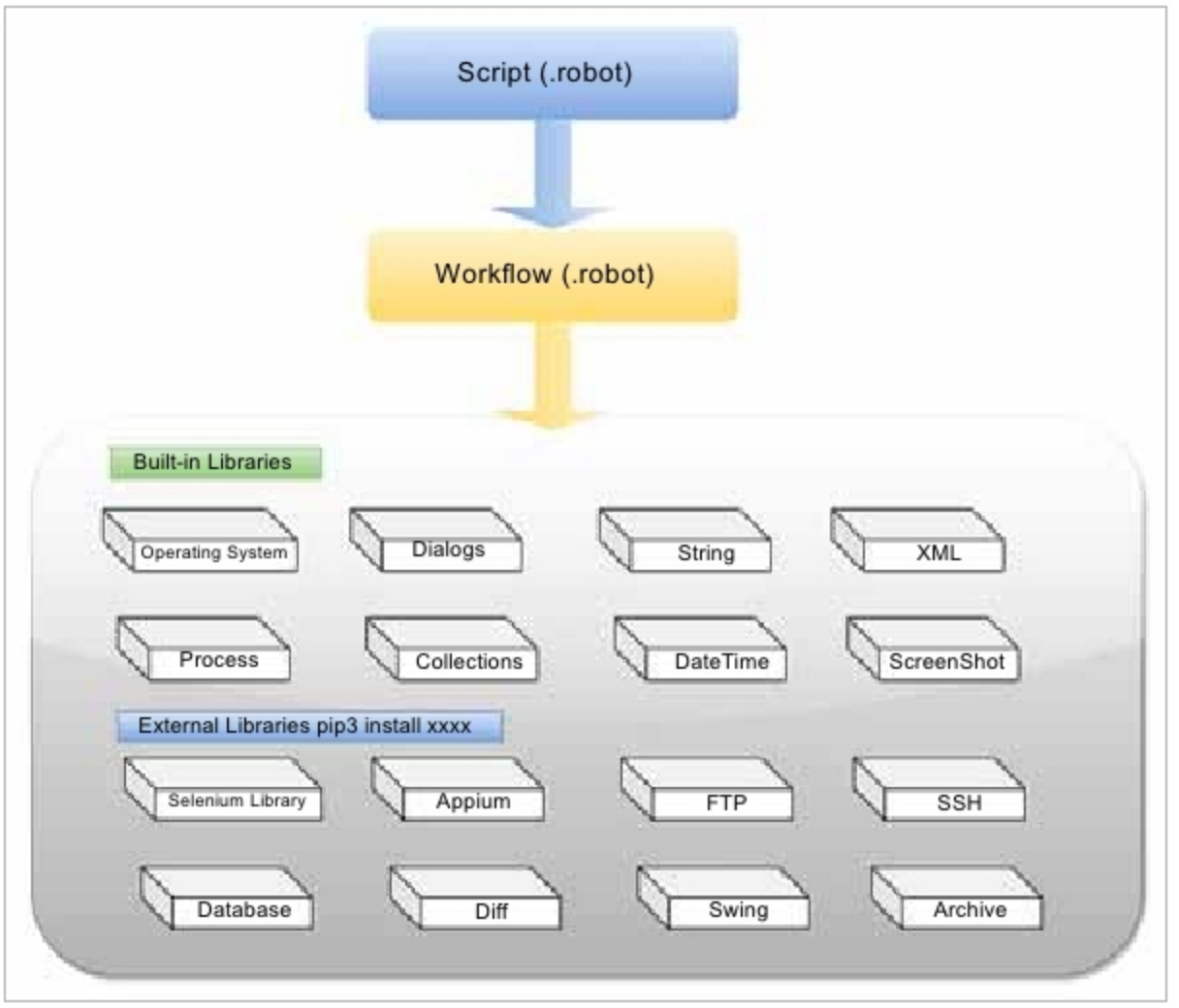

Figure 2: Robot Framework libraries

Installation steps on a Mac

- Install Python and PIP (add to your Path)
- Use PIP to install the Robot Framework
- Use PIP to install the Selenium library
- Select and install the desired browser versions
- Install Selenium WebDriver for each browser
- Install the PyCharm IDE and Intellibot plugin
- Create the base directory for the script

1. Install Python and PIP (add to your path) with the following command:

```
$ brew install python
```

Homebrew installs PIP pointing to the Homebrew'd Python 3.

2. Use PIP to install the Robot Framework as follows:

```
$ pip3 install robotframework
```

3. Use PIP to install the Selenium library:

```
$ pip3 install robotframework-
seleniumlibrary
```

4. Install Selenium WebDriver for each browser, e.g., install Chrome WebDriver for Chrome:

```
$ brew cask install chromedriver
```

5. Install the PyCharm IDE and Intellibot plugin, as follows:

```
Open Pycharm and Pycharm > Preferences
```

Figure 3: Pycharm preferences

Select *Plugin*, and then search and install *IntelliBot @SeleniumLibrary Patched*.

Organising project files

We can organise our project files by using a layered script approach. As represented in Figure 5, the script file contains test cases – the keywords files contain lower level keywords that are created to help the script do its work, and the page object files are a way to encapsulate locators and functionality to represent different pages in the system under test. So the script falls under a test directory, while the keywords and page objects together fall under resources. If we happen to get into creating our own libraries using Python, we could also consider creating a libraries directory to hold them. If instead, we had multiple products and tests, we could consider an approach by which inside our test directory we have a Product 1 directory, which might hold Feature 1, Feature 2, and so on. This would mean shorter scripts for specific areas of our product. Of course, this assumes a Product 2 directory, a Product 3 directory, and so on. Then, inside the